

Xarxes

PRÀCTICA 4

Mario VILAR

20392724

David Díez

20392643

27 de desembre de 2024

Professors: Manel López i Ferran Adelantado



UNIVERSITAT DE
BARCELONA

ÍNDEX

1	Introducció	2
2	Creació del canal recol·lector de dades	2
3	Simulació amb Packet Tracer	14
4	Conclusions	31

INTRODUCCIÓ

L'objectiu de la pràctica és conèixer el funcionament del protocol TCP/IP i treballar sobre aquesta pila per muntar un sistema de transferència de dades entre un dispositiu i un servidor, analitzant com són els paquets necessaris.

Per tal d'assolir l'objectiu, treballarem amb una doble vessant. Per una banda, la primera part de la pràctica estarà basada en els dispositius que hem fet servir a la pràctica anterior. Aquesta primera part de la pràctica es realitzarà amb l'ajut dels professors de pràctiques de l'assignatura.

Per altra banda, per tal d'analitzar el comportament dels diferents protocols, ens instal·larem un software de simulació que ens permetrà l'estructura dels diferents paquets que estem enviant per la xarxa. El simulador que farem servir es el Packet Tracer de Cisco. Aquesta part de la pràctica serà prèviament explicada pels professors de l'assignatura.

CREACIÓ DEL CANAL RECOL·LECTOR DE DADES

Observació 2.1. Seguint al peu de la lletra les instruccions de la pràctica obtenim el següent codi

```
1  #include <ESP8266WiFi.h>
2
3  // Wi-Fi Settings
4  const char* ssid = "XXX"; // your wireless network name (SSID)
5  const char* password = "XXXXXXXXXX"; // your Wi-Fi network password
6
7  WiFiClient client;
8
9  // ThingSpeak Settings
10 const int channelID = XXX;
11 String writeAPIKey = "XXXXXXXXXXXXXXXXXXXX"; // write API key for your ThingSpeak Channel
12 const char* server = "api.thingspeak.com";
13 const int postingInterval = 20 * 1000; // post data every 20 seconds
14
15 void setup() {
16 }
17
18 void loop() {
19     if (client.connect(server, 80)) {
20         // Measure Signal Strength (RSSI) of Wi-Fi connection
21         long rssi = WiFi.RSSI();
22         // Construct API request body
23         String body = "field1=";
```

```

24     body += String(rssi);
25     Serial.print("RSSI: ");
26     Serial.println(rssi);
27     client.println("POST /update HTTP/1.1");
28     client.println("Host: api.thingspeak.com");
29     client.println("User-Agent: ESP8266 (nothans)/1.0");
30     client.println("Connection: close");
31     client.println("X-THINGSPEAKAPIKEY: " + writeAPIKey);
32     client.println("Content-Type: application/x-www-form-urlencoded");
33     client.println("Content-Length: " + String(body.length()));
34     client.print(body);
35 }
36 client.stop();
37 delay(postingInterval);
38 }

```

però el provem i no ens acaba de funcionar com volem: no obtenim cap resposta al panel de ThingSpeak. Per això hem hagut d'aplicar una sèrie de canvis i ens ha quedat el següent codi

```

1  #include <ESP8266WiFi.h>
2
3  // Wi-Fi Settings
4  const char* ssid = "XXX"; // your wireless network name (SSID)
5  const char* password = "XXXXXXXXXX"; // your Wi-Fi network password
6
7  WiFiClient client;
8
9  // ThingSpeak Settings
10 const int channelID = XXX;
11 String writeAPIKey = "XXXXXXXXXXXXXXXXXXXX"; // write API key for your ThingSpeak Channel
12 const char* server = "api.thingspeak.com";
13 const int postingInterval = 20 * 1000; // post data every 20 seconds
14
15 void setup() {
16     Serial.begin(9600);
17     Serial.println();
18     Serial.println("Connectant a WiFi...");
19
20     WiFi.begin(ssid, password);
21     while (WiFi.status() != WL_CONNECTED) {
22         delay(1000);
23         Serial.print(".");
24     }
25     Serial.println("\nWiFi connectada!");
26     Serial.print("Direcció IP: ");

```

```

27     Serial.println(WiFi.localIP());
28 }
29
30 void loop() {
31     // Verifiquem que la WiFi estigui connectada
32     if (WiFi.status() == WL_CONNECTED) {
33         if (client.connect(server, 80)) {
34             // Measure Signal Strength (RSSI) of Wi-Fi connection
35             long rssi = WiFi.RSSI();
36
37             // Construct API request body
38             String body = "field1=" + String(rssi);
39             client.println("POST /update HTTP/1.1");
40             client.println("Host: api.thingspeak.com");
41             client.println("Connection: close");
42             client.println("X-THINGSPEAKAPIKEY: " + writeAPIKey);
43             client.println("Content-Type: application/x-www-form-urlencoded");
44             client.println("Content-Length: " + String(body.length()));
45             client.println();
46             client.print(body);
47
48             while (client.available()) {
49                 String line = client.readStringUntil('\r');
50                 Serial.print(line);
51             }
52             client.stop();
53         } else {
54             Serial.println("Error connectant-nos al servidor.");
55         }
56     } else {
57         Serial.println("WiFi desconnectat.");
58         WiFi.begin(ssid, password);
59     }
60
61     delay(postingInterval);
62 }

```

que essencialment és el mateix però té un parell de correccions. Per exemple, estem escrivint codi a `setup()` per tal de configurar el Serial Plotter i *connectar-nos a la WiFi donada*, cosa que en el codi anterior no s'intentava però és fonamental. A la funció `loop()` no executem res fins assegurar-nos que estem connectats. En cas contrari ens intentem connectar (veure línia 58).

Per tant, a partir d'ara ens referirem a aquest últim codi.

Exercici 2.2. *Expliqueu que fan el conjunt d'instruccions que tenim dins la sentència `if`.*

Resolució. Dins de la condició `if` tenim escrit

```

1  // Measure Signal Strength (RSSI) of Wi-Fi connection
2  long rssi = WiFi.RSSI();
3
4  // Construct API request body
5  String body = "field1=" + String(rssi);
6  client.println("POST /update HTTP/1.1");
7  client.println("Host: api.thingspeak.com");
8  client.println("Connection: close");
9  client.println("X-THINGSPEAKAPIKEY: " + writeAPIKey);
10 client.println("Content-Type: application/x-www-form-urlencoded");
11 client.println("Content-Length: " + String(body.length()));
12 client.println();
13 client.print(body);
14
15 while (client.available()) {
16     String line = client.readStringUntil('\r');
17     Serial.print(line);
18 }

```

que el que fa és el següent:

1. A la línia 4 obté la intensitat del senyal. Per això és fonamental assegurar-nos que estem connectats. Recordem que la idea del nostre codi és poder enviar aquesta dada a ThingSpeak, per tant hem de guardar aquest valor.
2. A la línia 5 construïm el cos de la sol·licitud HTTP que volem enviar. Com que el RSSI és un nombre, cal utilitzar `String()` per tal d'enviar-ho.
3. Les següents línies, des de la 6 fins la 13 el que fem és enviar el header del contingut al servidor. Observem que no estem fent `Serial.println()` com hem fet en altres ocasions sinó que cridem a `client.println()` on l'objecte `client` és del tipus `WiFiClient` que ens permet tenir comunicació amb ThingSpeak.
La primera instrucció especifica que estem fent una sol·licitud POST, és a dir, volem escriure dades. A la línia 7 indiquem que el servidor de destí és `api.thingspeak.com`. Un cop s'ha enviat la petició s'executa la línia 8 per demanar al servidor que tanqui la connexió. A la línia 9 hem d'enviar la nostra clau API per poder autenticar-nos. Les línies 10 i 11 acaben de configurar el header abans d'afegir un salt de línia i finalment a la línia 13 enviar la petició.
4. Al bucle `while` que comença a la línia 15 llegim i mostrem la resposta del servidor. ■

Exercici 2.3. Un cop explicada la funcionalitat del codi, expliqueu detalladament el seu funcionament i com interacciona amb el servidor de thingSpeak.

Resolució. La idea general del codi és establir comunicació amb el servidor de ThingSpeak utilitzant el protocol HTTP. El funcionament del codi en detall és el següent

1. Definició de les variables:

1. A les primeres línies del codi definim les variables necessàries per l'execució del programa. Trobem dades com el nom i la contrasenya de la WiFi on ens volem connectar i dades importants per establir connexió amb ThingSpeak. Definim `postingInterval` a 20.000 mil·lisegons, és a dir, la freqüència d'enviament és cada 20 segons.

2. Configuració inicial i connexió Wi-Fi:

1. La primera funció definida és `setup()` on comencem inicialitzant el Serial Plotter.
2. Acte seguit establim la connexió a una xarxa Wi-Fi mitjançant `WiFi.begin(ssid, password)`.
3. Comprovem periòdicament l'estat de la connexió amb `WiFi.status()` fins que es connecta correctament. Això és fonamental, ja que sense connexió el nostre codi no es pot comunicar amb ThingSpeak.

3. Obtenció del RSSI:

1. Un cop dins de la funció `loop()` i dins de la condició `if` (com ja hem explicat a l'exercici 2.2), utilitzem `WiFi.RSSI()` per obtenir la intensitat del senyal (RSSI). Aquest valor es guarda com un nombre enter i es convertirà en una cadena per ser enviat al servidor.

4. Construcció i enviament de la petició HTTP:

1. Primer construïm el cos de la sol·licitud HTTP, que conté la dada que es vol enviar (el RSSI), mitjançant `String body = "field1=" + String(rssi)`. El camp `field1` correspon al primer camp del canal de ThingSpeak.
2. A continuació, es construeix el header de la sol·licitud amb diverses línies de `client.println()`:
 1. Especifica que es tracta d'una sol·licitud POST al recurs `/update` del servidor.
 2. Proporciona el nom del servidor (`api.thingspeak.com`).
 3. Es defineix que es tancarà la connexió després d'enviar la sol·licitud (`Connection: close`).
 4. Es passa la clau API d'escriptura mitjançant `X-THINGSPEAKAPIKEY` per autenticar la petició.
 5. Es defineix el tipus de contingut (`application/x-www-form-urlencoded`) i la longitud del cos.
3. Finalment, s'envia una línia en blanc per separar el header del cos de la sol·licitud, i després s'envia el cos amb `client.print(body)`.

5. Resposta del servidor:

1. Després d'enviar la sol·licitud, el codi espera i llegeix la resposta del servidor mitjançant un bucle

while que processa línia per línia amb `client.readStringUntil('\r')`. Aquesta resposta normalment conté:

1. Un codi d'estat HTTP, com ara 200 OK, que indica que la sol·licitud ha estat processada correctament.
2. Informació addicional sobre l'estat del servidor i la dada processada.

6. Tancament de la connexió:

1. Un cop llegida la resposta del servidor, tanquem la connexió amb `client.stop()` per alliberar recursos i evitar connexions innecessàries.

La interacció del codi amb ThingSpeak consisteix en els següents passos:

1. El servidor de ThingSpeak rep la petició i comprova la clau API per autenticar l'usuari.
2. Si la petició és vàlida, ThingSpeak emmagatzema el valor del RSSI en el camp corresponent del canal associat.
3. El servidor actualitza el gràfic i el registre de dades del canal. Aquesta informació es pot visualitzar a la interfície web de ThingSpeak.
4. Si hi ha un error, el servidor retorna un codi d'estat diferent (com 400 Bad Request o 403 Forbidden), que s'hauria de gestionar adequadament al codi. Són casuístiques que tenim controlades a les línies 32 i 33 amb `WiFi.status()` i `client.connect(server, 80)`. ■

Una altra manera de fer-ho és utilitzant la llibreria `ThingSpeak.h` proporcionada per la plataforma i que facilita tota la connexió.

Exercici 2.4. Torna a resoldre les qüestions 1 i 2 però ara amb la llibreria de ThingSpeak. Si implementes les dues, quines diferències hi trobes?

Resolució. Ara hem descarregat la llibreria `ThingSpeak.h` i hem adaptat el nostre codi per tal d'obtenir

```

1  #include <ESP8266WiFi.h>
2  #include <ThingSpeak.h>
3
4  // Wi-Fi Settings
5  const char* ssid = "XXX"; // your wireless network name (SSID)
6  const char* password = "XXXXXXXXXX"; // your Wi-Fi network password
7
8  WiFiClient client;
9
10 // ThingSpeak Settings
11 const int channelID = XXX;
12 String writeAPIKey = "XXXXXXXXXXXXXXXXXXXX"; // write API key for your ThingSpeak Channel
13

```

```
14 const int postingInterval = 20 * 1000; // post data every 20 seconds
15
16 void setup() {
17     Serial.begin(9600); // Inicialitza el monitor sèrie
18     Serial.println("Inicialitzant...");
19
20     // Connectar-se a la Wi-Fi
21     WiFi.begin(ssid, password);
22     Serial.print("Connectant a Wi-Fi");
23     while (WiFi.status() != WL_CONNECTED) {
24         delay(1000);
25         Serial.print(".");
26     }
27     Serial.println("\nConnexió establerta!");
28     Serial.print("Direcció IP: ");
29     Serial.println(WiFi.localIP());
30
31     // Inicialitzar ThingSpeak
32     ThingSpeak.begin(client); // Vincula el client Wi-Fi amb ThingSpeak
33 }
34
35 void loop() {
36     if (WiFi.status() == WL_CONNECTED) {
37         // Obtenir el valor del RSSI (intensitat del senyal)
38         long rssi = WiFi.RSSI();
39         Serial.print("RSSI: ");
40         Serial.println(rssi);
41
42         // Escriure la dada al camp 1 de ThingSpeak
43         int status = ThingSpeak.writeField(channelID, 1, rssi, writeAPIKey);
44
45         // Verificar si l'operació ha estat exitosa
46         if (status == 200) {
47             Serial.println("Dada enviada correctament a ThingSpeak!");
48         } else {
49             Serial.print("Error enviant dada. Codi: ");
50             Serial.println(status);
51         }
52     } else {
53         Serial.println("Wi-Fi desconectat. Intentant reconnectar...");
54         WiFi.begin(ssid, password);
55     }
56
57     delay(postingInterval);
58 }
```

on podem trobar diferències a totes parts del codi:

1. A la definició de variables ja que ara no ens cal tenir el `server`.
2. La funció `status()` és quasi idèntica però ara hem hagut d'afegir la línia 30 per establir connexió entre el client i `ThingSpeak`.
3. La diferència més gran la trobem a la funció `loop()` on ens estalviem moltíssim codi. Ara tan sols hem de comprovar que la connexió amb la WiFi sigui vàlida i utilitzem la funció `writeField` de la llibreria `ThingSpeak.h` per fem l'enviament de dades directament en una sola línia. Aquesta línia ens retorna el valor `status` de la petició, en cas de ser 200 vol dir que tot ha anat bé.

La implementació del codi amb `ThingSpeak.h` ens ha aportat molt a l'hora d'enviar les dades cap a `ThingSpeak`.

Contestem ara a la pregunta 2.2 però aplicada a aquest codi.

Dins de la condició `if` tenim escrit:

```

1  // Obtenir el valor del RSSI (intensitat del senyal)
2  long rssi = WiFi.RSSI();
3  Serial.print("RSSI: ");
4  Serial.println(rssi);
5
6  // Escriure la dada al camp 1 de ThingSpeak
7  int status = ThingSpeak.writeField(channelID, 1, rssi, writeAPIKey);
8
9  // Verificar si l'operació ha estat exitosa
10 if (status == 200) {
11     Serial.println("Dada enviada correctament a ThingSpeak!");
12 } else {
13     Serial.print("Error enviant dada. Codi: ");
14     Serial.println(status);
15 }
```

Aquestes instruccions realitzen les següents accions:

1. A la línia 2 obtenim el valor del RSSI utilitzant `WiFi.RSSI()` i mostrem aquest valor al monitor sèrie.
2. A la línia 7 escrivim el valor de RSSI al camp 1 del canal de `ThingSpeak` utilitzant la funció `ThingSpeak.writeField`. Aquesta funció accepta com a paràmetres:
 1. El `channelID` del canal de `ThingSpeak` on volem emmagatzemar la dada.
 2. El número de camp on es guardarà el valor (1 en aquest cas).
 3. El valor del RSSI obtingut a la línia 2.
 4. La clau d'API (`writeAPIKey`) que autentica l'operació.
3. A la línia 10, verifiquem si l'operació ha estat exitosa comprovant el codi de retorn:
 1. Si el codi és 200, significa que les dades s'han enviat correctament, i ho notifiquem al monitor

sèrie.

2. Si el codi és diferent, imprimim el codi d'error per ajudar a identificar el problema.

Contestem ara a la pregunta 2.3 aplicada a aquest codi.

La idea general del codi és establir una comunicació amb el servidor de ThingSpeak per enviar dades utilitzant la llibreria `ThingSpeak.h`, que simplifica molt el procés. El funcionament del codi en detall és el següent:

1. Definició de les variables:

1. A les primeres línies del codi definim les credencials de la Wi-Fi (`ssid` i `password`), així com les dades del canal de ThingSpeak: `channelID` i `writeAPIKey`.

2. Configuració inicial i connexió Wi-Fi:

1. A la funció `setup()` inicialitzem el monitor sèrie i comença la connexió Wi-Fi mitjançant `WiFi.begin(ssid, password)`.
2. Comprovem periòdicament l'estat de la connexió amb `WiFi.status()` fins que s'estableix correctament. Mostrem al monitor sèrie l'adreça IP assignada.
3. Finalment, inicialitzem la connexió amb ThingSpeak mitjançant `ThingSpeak.begin(client)`, que vincula el client Wi-Fi amb la llibreria ThingSpeak. Aquesta línia és diferent entre el codi que utilitza la llibreria `ThingSpeak.h` i el que no.

3. Obtenció del RSSI i enviament a ThingSpeak:

1. Dins de la funció `loop()` i dins de la condició `if`, utilitzem `WiFi.RSSI()` per obtenir la intensitat del senyal Wi-Fi. Aquest valor es mostra al monitor sèrie.
2. Utilitzem la funció `ThingSpeak.writeField` per enviar el valor del RSSI al camp 1 del canal de ThingSpeak. La funció encapsula automàticament la petició HTTP, incloent-hi el cos i els encapçalaments, i retorna un codi d'estat HTTP que indica el resultat de l'operació.

4. Verificació de l'estat de l'operació:

1. Comprovem si el codi de retorn és 200, que indica que l'operació ha estat exitosa, i mostrem un missatge confirmant-ho al monitor sèrie.
2. Si el codi no és 200, mostrem un missatge d'error amb el codi rebut per ajudar en la depuració.

5. Gestió de la connexió Wi-Fi:

1. Si en qualsevol moment la connexió Wi-Fi es perd (`WiFi.status() != WL_CONNECTED`), es reintenta la connexió amb `WiFi.begin(ssid, password)`.

6. Interval d'enviament:

1. El codi espera 20 segons abans d'enviar el següent valor de RSSI, complint amb les normes de ThingSpeak per no sobrecarregar el servidor.

La interacció amb ThingSpeak consisteix en els següents passos:

1. El servidor de ThingSpeak rep la sol·licitud i valida la clau d'API d'escriptura `writeAPIKey`.
2. Si la petició és vàlida, ThingSpeak emmagatzema el valor del RSSI al camp `i` del canal associat `channelID`.
3. El servidor actualitza els gràfics i registres del canal a la interfície web de ThingSpeak.
4. En cas d'error, ThingSpeak retorna un codi d'estat que ajuda a identificar el problema (p. ex., 403 Forbidden per clau API incorrecta).

Exercici 2.5. *Veieu que efectivament, es mostra a la web de ThingSpeak la vostra dada. Feu una captura i afegiu-lo a la vostra entrega.*

Demostració. A la secció *channels* dintre de la web de ThingSpeak podem veure tots els canals que tenim, en el nostre cas només és un. Fem clic sobre ell i podem accedir a les dades. En particular, la secció *Private View* podem veure la taula amb el gràfic generat a partir de les RSSI que hem enviat des de la mota.

La captura de pantalla provant això és:

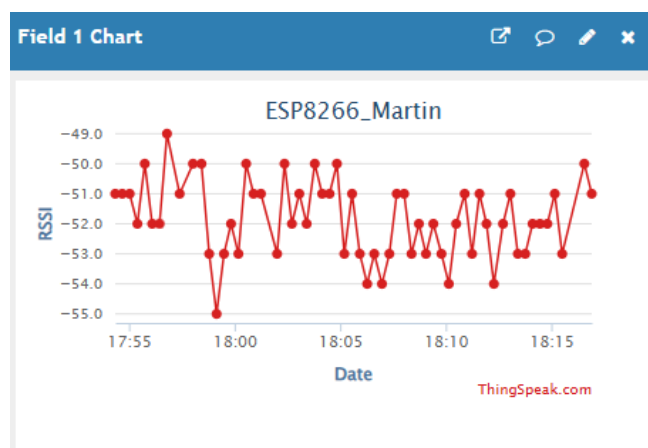


Figura 1: Visualització de les dades

Exercici 2.6. *Genera ara un programa en Python per analitzar les dades. Simplement, descarrega el conjunt de dades generades per la nostra mota durant un període mínim de una hora, calculeu el valor mitjà de la RSSI així com la desviació estàndard.*

Resolució. Podem descarregar les dades des de la part de *Private View* fent clic al botó de *Export recent data*. Se'ns obrirà un menú on pregunta quines dades volem i en quin format les volem. En cas de voler descarregar totes les dades de tots els paràmetres hem de clicar a la opció que diu `ESP8266_Martin Channel Feed`. Si només volem les dades d'un paràmetre en particular hem de clicar a una opció de l'estil `Field 1 Data: RSSI`. En el nostre cas com que tan sols estem estudiant un paràmetre (el RSSI) no importa quina de les opcions escollim ja que seran iguals.

Pel que fa el tipus de fitxer, tenim la opció de descarregar un JSON, XML o CSV. En el nostre cas hem descarregat el format CSV per tal de poder analitzar les dades utilitzant pandas de Python.

Hem fet diverses proves fent descàrregues seguint aquest procés però tan sols obtenim les últimes dades, no el registre sencer i com que l'enunciat demana dades d'almenys 1 hora, això no serveix.

Un altre mètode per obtenir les dades és fer-ho des de la secció *Data Import/Export*. Aquí, a la part superior hi ha un botó gran de color verd que diu *Download*. Tan sols fem clic i obtenim el fitxer `feeds.csv` amb totes les dades històriques. A més ens permet seleccionar la franja horària que volem utilitzar, en el nostre cas (GMT+01:00) Madrid.

Les primeres línies del fitxer són:

created at	entry id	field1	latitude	longitude	elevation	status
2024-12-21T16:07:28+01:00	1	-43				
2024-12-21T16:10:03+01:00	2	-41				
2024-12-21T16:11:06+01:00	3	-35				

on la columna `field1` conté les dades del RSSI. Fent una petit ullada a la taula podem veure que tenim 467 registres des de les 16:07:28 fins les 19:15:54.

El nostre codi Python per trobar la mitjana i la desviació estàndard és

```

1  # Importem pandas per fer l'anàlisi
2  import pandas as pd
3
4  # Carreguem les dades que hem descarregat de ThingSpeak
5  df = pd.read_csv("feeds.csv")
6  display(df)
7
8  # Guardem la columna amb les dades del RSSI
9  rssi = df["field1"]
10
11 # Calculem la mitjana amb 2 decimals
12 mean = rssi.mean()
13 print(f"La mitjana d'aquesta mostra és {round(mean, 2)} dBm")
14
15 # Calculem la desviació estàndard amb 2 decimals
16 std = rssi.std()
17 print(f"La desviació estàndard d'aquesta mostra és {round(std, 2)} dBm")

```

El resultat obtingut és que la mitjana és de $-51,15$ dBm i la desviació estàndard de $2,10$ dBm. ■

Exercici 2.7. *Explica i analitza el comportament complert del sistema desenvolupat: Els generadors de senyal RSSI i el recopilador de dades.*

Resolució. El sistema desenvolupat es compon de dues parts principals:

1. **Generadors de senyal RSSI:** El dispositiu ESP8266 mesura periòdicament la intensitat del senyal Wi-Fi utilitzant la funció `WiFi .RSSI ()`. Aquesta dada reflecteix la qualitat de la connexió Wi-Fi i es mesura en dBm.
2. **Recopilador de dades:** Les dades del RSSI es recullen i s'envien a la plataforma ThingSpeak mitjançant peticions HTTP (manuals o amb la llibreria `ThingSpeak .h`), on es guarden al canal corresponent per a l'emmagatzematge i l'anàlisi.

El RSSI és una mesura logarítmica de la intensitat del senyal Wi-Fi rebuda, recollida cada 20 segons. Els resultats mostren una mitjana de $-51,15$ dBm, indicant bona qualitat de senyal, amb una desviació estàndard de $2,10$ dBm, que reflecteix baixa variabilitat i un entorn estable.

L'ESP8266 envia els valors del RSSI a ThingSpeak mitjançant peticions HTTP, simplifícades amb la llibreria `ThingSpeak .h`. Les dades es guarden al canal de ThingSpeak i es poden analitzar en format gràfic o numèric per identificar patrons i possibles problemes de xarxa.

El sistema és estable gràcies a la baixa variabilitat del RSSI i és útil per monitoritzar la qualitat de la xarxa Wi-Fi. No obstant això, factors externs com la distància o obstacles poden afectar els resultats. ■

Fins ara, hem treballat amb el nostre mòdul configurant-lo com a client per tal que es connecti a una xarxa Wi-Fi. En aquest exercici el que farem es que sigui el nostre mòdul qui generi una xarxa Wi-Fi amb la qual ens podem connectar i transmetre informació.

Per configurar el mòdul ESP8266 com a un punt d'accés hem de passar de la configuració normal a la de `WIFI_AP`. Això es fa utilitzant la comanda `WiFi .mode (WIFI_AP)`.

Farem servir la funció `softAP` per tal de passar el nom i la contrasenya de la Wi-Fi que volem crear. Amb això ja tindrem la nostra Wi-Fi funcionant i ens podrem connectar a ella des de qualsevol dispositiu, per exemple altres motes.

Exercici 2.8. *A partir de tot això, munteu una arquitectura client-servidor, on el client fa una petició de dades i el servidor les subministra. Per exemple, el servidor pot ser la Mota que també genera la wifi a la que es connectarà el client. Teniu llibertat per escollir que i com transmetre. Per exemple podeu demanar un valor aleatori, o muntar un xat fent servir la consola... Transmeteu les dades entre client i servidor fent servir un fitxer JSON.*

1. *Explica qui és, en el teu cas, el client i qui és el servidor. Per què?*
2. *Quina mida té el teu fitxer JSON? Genera un fitxer JSON de 10k Bytes. Fragmenta'l en paquets de 1k Byte i transmet-los de tal manera que les dades arribin correctament.*

Resolució.

1. El servidor és qui espera les sol·licituds i respon amb la informació sol·licitada. El client, en canvi, és qui inicia la comunicació sol·licitant dades al servidor. Per tant en el sistema que desenvoluparem tenim

1. **Servidor:** L'ESP8266 actua com a servidor, ja que és responsable de crear la xarxa Wi-Fi i respondre a les sol·licituds HTTP. A més, subministra dades en format JSON a través de la ruta configurada.
2. **Client:** Qualsevol dispositiu connectat a la xarxa creada per l'ESP8266, com ara un telèfon mòbil, un ordinador o un altre ESP8266, que faci una sol·licitud HTTP al servidor.
2. El fitxer JSON generat té una mida exacta de 10 kB i conté informació sobre personatges de Breaking Bad en format clau-valor. Els passos per transmetre'l són:
 1. Fragmentar el fitxer en paquets de 1 kB utilitzant l'índex de cada paquet.
 2. Enviar cada paquet al client com una resposta HTTP a una sol·licitud a la ruta /paquet.
 3. Els clients poden especificar l'índex del paquet desitjat mitjançant un paràmetre en la sol·licitud (index).

El codi corresponent a aquesta funcionalitat es troba implementat en la funció `handlePaquet`, on es llegeixen els paràmetres del client, es calcula la mida del paquet corresponent i es retorna com a resposta HTTP. ■

Observació 2.9. Com hem dit fa un moment, fins ara era al revés; és a dir, la nostra mota estava configurada com a client per tal que es connectés a una xarxa WiFi i pugés dades al servidor, en el nostre cas el de ThinkSpeak.

3

SIMULACIÓ AMB PACKET TRACER

Assigneu a cada PC una IP privada del tipus 192.168.0.x amb màscara 255.255.255.0, fent servir DHCP. Reserva les 10 primeres adreces d'aquesta xarxa per futures millores. Penja aquesta xarxa de la interfície del router FA0/0. Assigneu a aquesta interfície la IP 192.168.0.254.

Exercici 3.1. *Quina comanda fem servir per reservar 10 adreces?*

Demostració. Si volem reservar 10 adreces, utilitzarem la comanda `ip dhcp excluded-address` per deixar algunes IP per escollir de forma fixa. Nosaltres tenim la xarxa 192.168.0.0. Per a fer-ho executem:

```
1 Router(config)#ip dhcp excluded-address 192.168.0.2 192.168.0.10
```

L'adreça 192.168.0.1 la tenim al router i per tant desitgem no assignar dinàmicament les adreces que van des del 192.168.0.2 al 192.168.0.10. En més detall, hem de configurar adequadament router, switch i PCs. No entrarem en molt detall, ja que vam fer un procediment anàleg a la pràctica anterior. Si hi ha algun dubte, es pot consultar aquell document. Resumint, el que fem és configurar tots els PCs en DHCP (tant el port FA0 com el gateway), connectem els diferents dispositius amb el raig, etc.

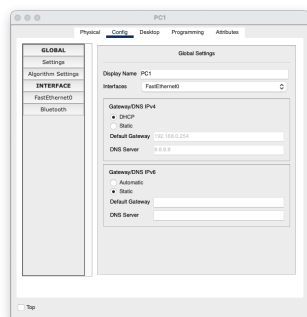


Figura 2: Configuració global de l'ordinador.

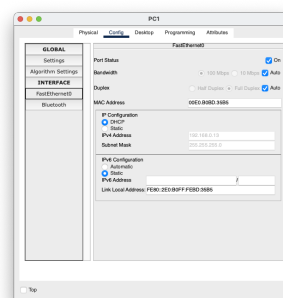


Figura 3: Configuració del FA0 de l'ordinador.

En particular, les comandes utilitzades per a la configuració del router han sigut aquestes. El raig ha connectat el Fa4 del router amb el Fa4 del Switch (en lloc del Fa0) per dues raons:

- Si connectem el switch amb els 4 PCs i després amb el router, omplirem els Fa0, . . . , Fa3, pel que connectarem amb el router amb el Fa4. Aquest port, PT-SWITCH-NM-1FFE, està destinat per a usar amb fibra òptica: *The PT-SWITCH-NM-1FFE Module provides one Fast-Ethernet interface for use with fiber media*. Pel que tipus de cable utilitzat canvia (veiem que apareix en vermell).
- Si ho intentem al revés; és a dir, si connectem el switch amb el router i després els PCs, d'una banda la connexió amb el router passaria a ser a través de Fa0 amb cable de coure (color negre) com al dibuix que ens proporciona el professorat. D'altra banda, el Packet Tracer no ens deixarà connectar l'últim ordinador (*the cable cannot be connected to that port*). Això és perquè solament quedarien disponibles els ports amb fibra òptica, però els PCs no disposen de tal port. La solució seria *afegir un altre port PT-SWITCH-NM-1FFE al switch*.

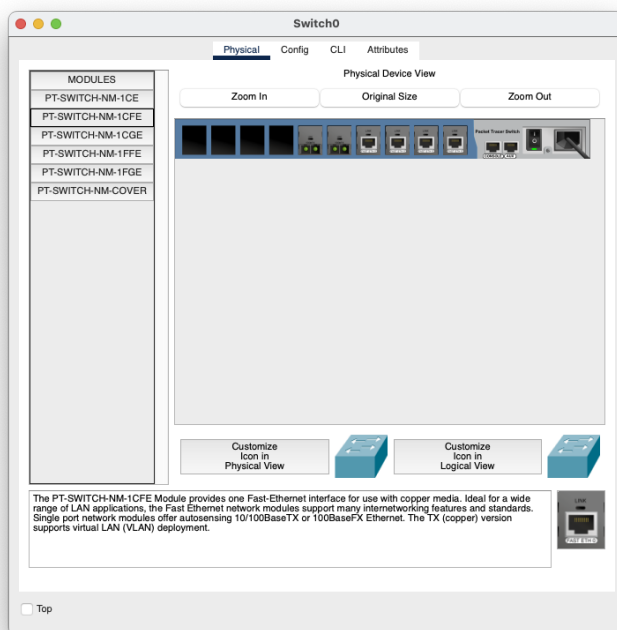


Figura 4: Afegir un altre port al Switch0.

Nosaltres hem optat per seguir la solució del primer ítem, ja que no influeix en el resultat final i, a més, va ser la primera que vam trobar. La segona ha estat a posteriori, després de l'implementació de la xarxa.

```

1 Router(config)#ip dhcp excluded-address 192.168.0.2 192.168.0.10
2 Router(config)#ip dhcp pool NET
3 Router(dhcp-config)#network 192.168.0.0 255.255.255.0
4 Router(dhcp-config)#default-router 192.168.0.254
5 Router(dhcp-config)#dns-server 8.8.8.8
6 Router(dhcp-config)#exit
7 Router(config)#int Fa4/0
8 Router(config-if)#ip helper-address 192.168.0.254
9 Router(config-if)#exit
10 Router(config)#exit
11 Router#
12 %SYS-5-CONFIG_I: Configured from console by console

```

Executant la comanda `tracert 192.168.0.13` des del PC0:

```

1 C:\>tracert 192.168.0.13
2
3 Tracing route to 192.168.0.13 over a maximum of 30 hops:
4
5 1    0 ms    0 ms    0 ms    192.168.0.13

```

6
7 *Trace complete.*

Així doncs:

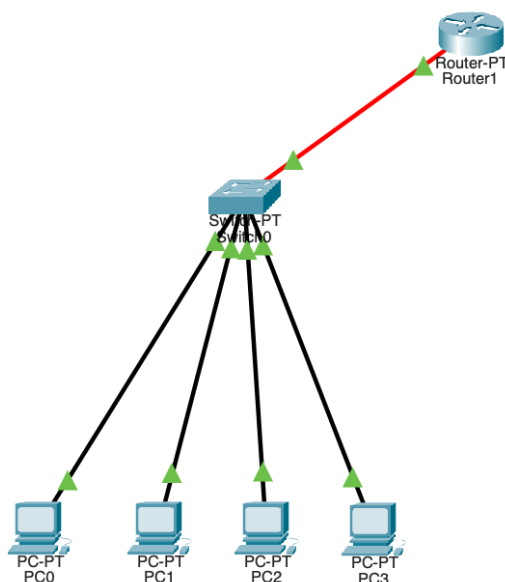


Figura 5: Router parcial.

I aquesta serà la configuració fins el moment, abans de passar al següent exercici. ■

A l'altra interfície del router posareu un punt d'accés sense fils i connectareu de forma remota 2 dispositius mitjançant DHCP. La xarxa que muntareu té la IP 192.168.1.0/24.

Exercici 3.2. *Fes una captura de pantalla on es mostri que hi ha connexió entre les dues interfícies del router explicant el procés de connexió.*

Demostració. El procés de connexió amb Tracer Route és anàleg al de la xarxa anterior. Amb el raig anem trobant el cablejat necessari per connectar les components d'aquesta segona xarxa; aquest cablejat serà tot de coure (cable negre) i penjarà de la interfície Fa1 del router. La configuració d'aquesta interfície serà molt similar a la que ja havíem fet, simplement caldrà tenir en compte que treballarem sobre una altra xarxa.

```
1 Router>enable
2 Router#config terminal
3 Router(config)#ip dhcp pool NET2
4 Router(dhcp-config)#network 192.168.1.0 255.255.255.0
5 Router(dhcp-config)#default-router 192.168.1.1
6 Router(dhcp-config)#dns-server 8.8.8.8
7 Router(dhcp-config)#exit
```

```

8 Router(config)#int Fa0/0
9 Router(config-if)#ip helper-address 192.168.1.1
10 Router(config-if)#ip address 192.168.1.1 255.255.255.0
11 Router(config-if)#exit
12 Router(config)#exit
13 Router#
14 %SYS-5-CONFIG_I: Configured from console by console

```

Notem que l'enunciat no ens demana excloure cap adreça de l'assignació automàtica per dhcp.

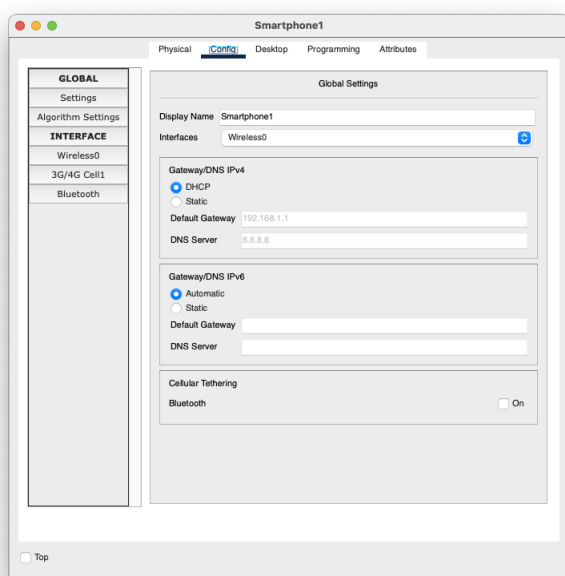


Figura 6: Configuració global de l'smartphone.

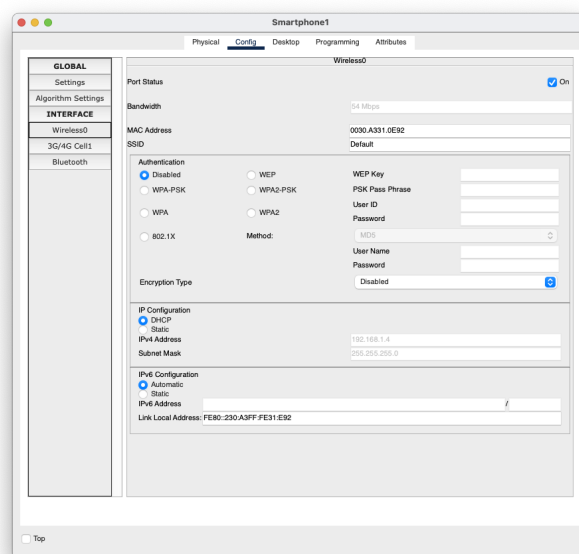


Figura 7: Configuració de l'interfície inalàmbrica de l'smartphone.

En el cas que ara ens ocupa dispondrem de tres dispositius que no havíem vist encara: un *access point* i dos *smartphones*. La connexió amb els dos dispositius mòbils serà inalàmbrica. Finalment, efectivament hi ha connexió entre les dues interfícies del router. Per exemple, des del PC1 podem trobar un dels smartphones:

```

1 C:\>tracert 192.168.1.3
2
3 Tracing route to 192.168.1.3 over a maximum of 30 hops:
4
5  1  0 ms      0 ms      0 ms      192.168.0.254
6  2  9 ms      19 ms     11 ms     192.168.1.3
7
8 Trace complete.

```

Finalment, el dibuix de la xarxa queda:

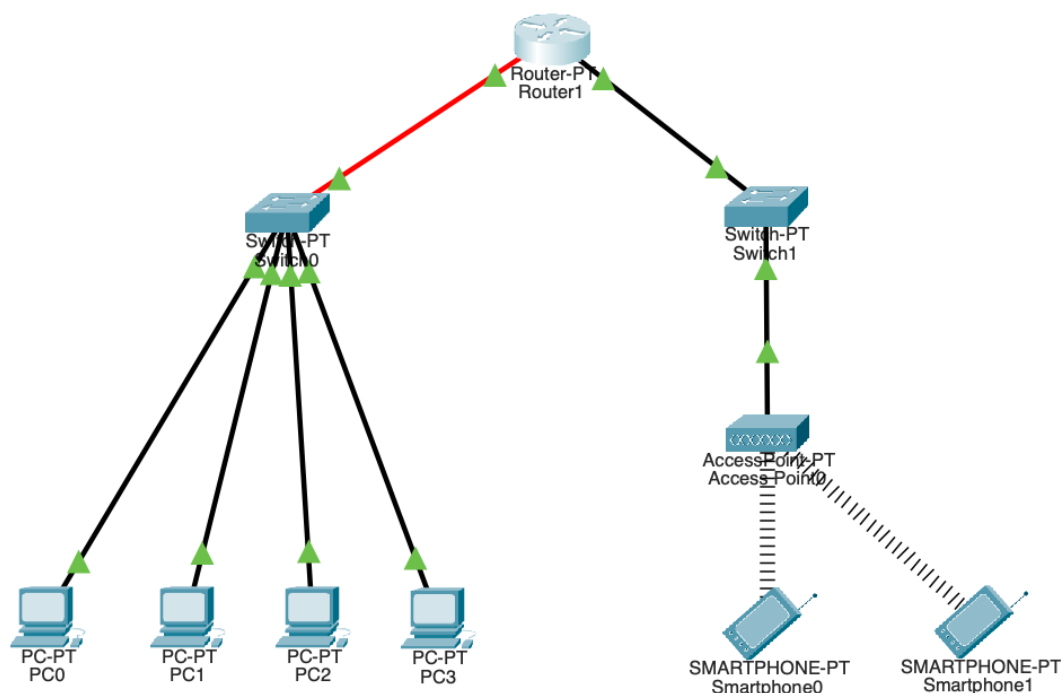


Figura 8: Xarxa completa.

Afegiu ara un nou router. Amb l'ajuda del professor, feu subnetting en les dues interfícies Fast Ethernet. Configureu una amb la IP de xarxa 192.168.2.0/25 i l'altra amb la IP 192.168.2.128/25. Connecteu a totes dues interfícies dos switches (un per cada interfície) i en aquests dos PCs. Configureu en tots dos casos DHCP.

Exercici 3.3. *Per a què serveix el subnetting? Quantes IPs hi ha disponible per cada subxarxa?*

Demostració. El subnetting té una sèrie de funcionalitats molt importants en el context de la creació de xarxes. D'una banda, evita el malbaratament d'adreces en xarxes grans on no calen moltes IPs. D'altra banda, millora del rendiment reduint el trànsit de difusió, limitant-lo a cada subxarxa. Pel que fa a la seguretat, aïlla les xarxes, evitant que el trànsit d'una subxarxa afecti les altres. I per últim, facilita la gestió organitzant els dispositius en subxarxes més petites i manejables.

La màscara /25 (11111111.11111111.11111111.10000000 o 255.255.255.128) s'emporta un bit més com a bit de xarxa i divideix una xarxa classe C en dues subxarxes amb 128 adreces per subxarxa:

1. 1 adreça per a la IP de xarxa (primera IP, ex.: 192.168.2.0 o 192.168.2.128).
2. 1 adreça per a la broadcast (última IP, ex.: 192.168.2.127 o 192.168.2.255).
3. 126 adreces disponibles per a dispositius (hosts) en cadascuna de les subxarxes.

Exercici 3.4. *Hi ha visibilitat entre les dues interfícies? En algun moment surt la IP 169.254.X.Y? Què vol dir aquesta adreça?*

Resolució. Hem de distingir dos casos, abans i després de configurar les interfícies del router en detall i crear les xarxes corresponents. Si abans de configurar la interfície connectem els dispositius amb el raig i ho configurem tot a DHCP, podem veure que els ordinadors mostren la següent configuració de la seva Fa0:

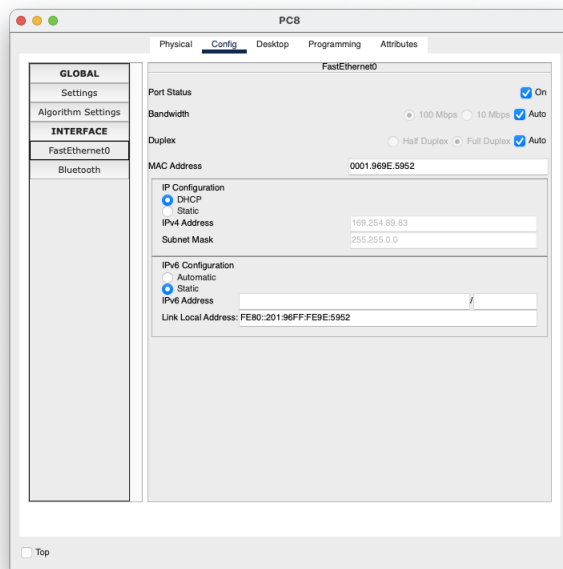


Figura 9: Configuració de la interfície de xarxa del pc8.

Quan no hi ha DHCP ni configuració manual, un dispositiu s'assigna una IP 169.254.X.Y per poder comunicar-se amb altres dispositius de la mateixa xarxa local (que també hagin rebut una IP d'aquest rang). Sol ser un senyal que hi ha un problema amb el servidor DHCP (no està disponible o hi ha un error de connexió). I, efectivament, hi ha un problema amb el servidor DHCP perquè encara no l'hem configurat. Executem el següent codi:

```

1 Router>enable
2 Router#config terminal
3 Router(config)#ip dhcp pool NET1
4 Router(dhcp-config)#network 192.168.2.0 255.255.255.128
5 Router(dhcp-config)#default-router 192.168.2.1
6 Router(dhcp-config)#dns-server 8.8.8.8
7 Router(dhcp-config)#exit
8 Router(config)#int Fa0/0
9 Router(config-if)#ip helper-address 192.168.2.1
10 Router(config-if)#ip address 192.168.2.1 255.255.255.128
11 Router(config-if)#exit
12 Router(config)#exit
13 Router#
14 %SYS-5-CONFIG_I: Configured from console by console

```

I aquest codi per l'altra interfície:

```

1 Router>enable
2 Router#config terminal
3 Router(config)#ip dhcp pool NET2
4 Router(dhcp-config)#network 192.168.2.128 255.255.255.128
5 Router(dhcp-config)#default-router 192.168.2.129
6 Router(dhcp-config)#dns-server 8.8.8.8
7 Router(dhcp-config)#exit
8 Router(config)#int Fa1/0
9 Router(config-if)#ip helper-address 192.168.2.129
10 Router(config-if)#ip address 192.168.2.129 255.255.255.128
11 Router(config-if)#exit
12 Router(config)#exit
13 Router#
14 %SYS-5-CONFIG_I: Configured from console by console

```

Si tornem a la mateixa pestanya del pc8 trobem que:

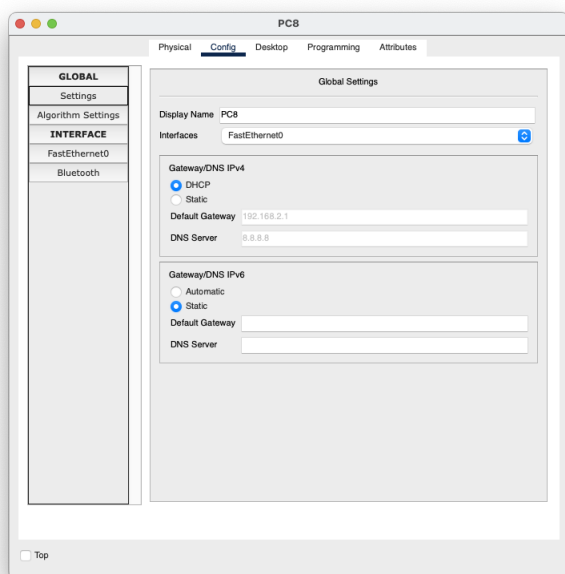


Figura 10: Configuració global de pc8.

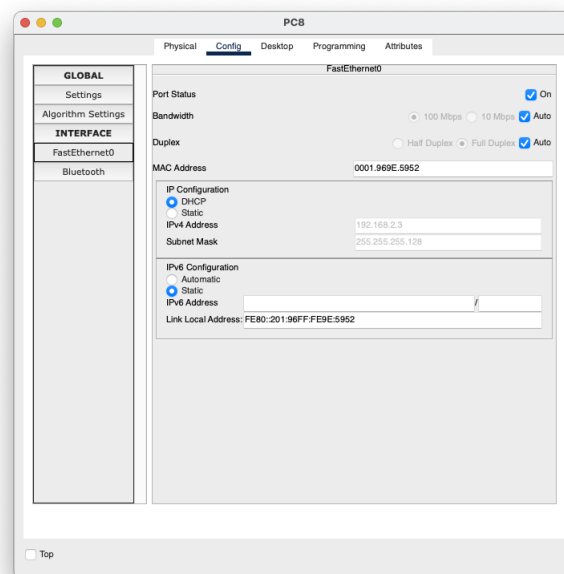


Figura 11: Configuració de l'interfície inalàmbrica de pc8.

Ara les IPs mostrades tenen molt més sentit amb allò que s'esperava. ■

Connecta ara els dos routers entre ells pel port sèrie. Els dos routers requeriran de un protocol d'encaïminament. Feu-lo manualment, amb la comanda iproute. Feu servir la IP pública 161.116.95.0/24. Si voleu, podeu implementar el protocol RIP. Trobareu informació de tots dos casos a Internet.

Exercici 3.5. *Explica detalladament quin és el mètode implementat i mostra el seu funcionament.*

Resolució. Unim Router1 i Router0 amb el raig, de manera que la connexió serà amb serial. És important tenir en compte que un dels extrems del cable (el costat DCE) necessitarà un paràmetre de sincronització que configurarem més tard.

1. Al Router1, obrim la configuració (fent clic al router) i anem a la pestanya Config. A la interfície sèrie corresponent, en el nostre cas Serial2/0, assignem l'adreça IP 161.116.95.1 amb una màscara 255.255.255.0. Com que és el costat DCE, també configurarem el clock rate per establir la velocitat de sincronització a 64000.
2. Al Router0, fem un procés similar, però assignem l'adreça IP 161.116.95.2. Aquí no caldrà configurar el clock rate, ja que aquest router és el costat DTE (receptor de la sincronització).
3. Comprovem que estiguin en estat up/up a través de les eines de verificació del Packet Tracer.

Un cop connectats, hem de donar una adreça IP a cada interfície sèrie perquè puguin comunicar-se. A cada router, anirem a la configuració de la interfície sèrie. Al Router1, obrim la configuració (fent clic al router) i anem a la pestanya CLI o Config. A la interfície sèrie corresponent, assignem l'adreça IP 161.116.95.1 amb una màscara 255.255.255.0. Com que és el costat DCE, també configurarem el clock rate per establir la velocitat de sincronització. Al Router0, fem un procés similar, però assignem l'adreça IP 161.116.95.2. Aquí no caldrà configurar el clock rate, ja que aquest router és el costat DTE (receptor de la sincronització).

Ara que les interfícies sèrie estan connectades i funcionals, necessitem establir com cada router aprendrà a arribar a les xarxes de l'altre. Ho farem configurant rutes estàtiques. Al Router1 utilitzem:

```
1 Router(config)#ip route 192.168.2.0 255.255.255.128 161.116.95.2
2 Router(config)#ip route 192.168.2.128 255.255.255.128 161.116.95.2
```

Al Router0 utilitzem:

```
1 Router(config)#ip route 192.168.0.0 255.255.255.0 161.116.95.1
2 Router(config)#ip route 192.168.1.0 255.255.255.0 161.116.95.1
```

La configuració del Router0 (al Router1 és totalment anàloga, tal com hem comentat) hauria de ser semblant a la següent:

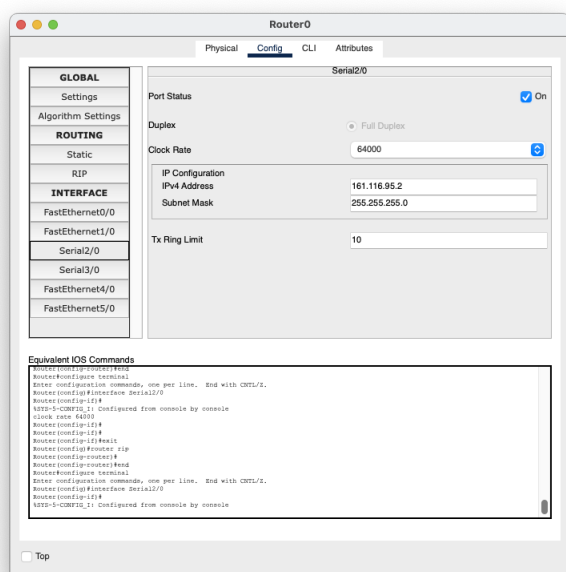


Figura 12: Configuració del port Serial2 del Router.

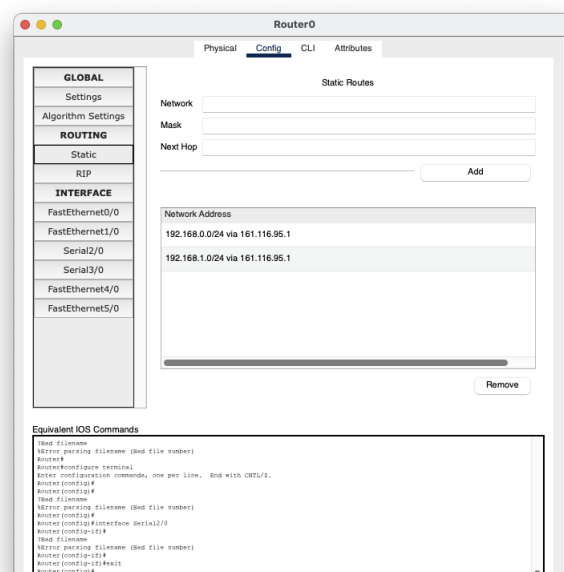


Figura 13: Configuració Router.

L'esquema resultant és el següent:

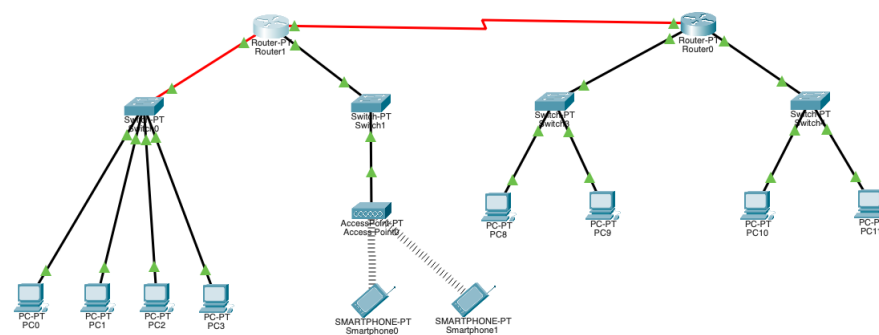


Figura 14: Esquema total.

Al problema següent se'ns demana demostrar que la connexió en serial entre els routers és exitosa i podem accedir entre qualssevol dispositius. Optarem per acabar aquesta resolució utilitzant una de les eines natives del Packet Tracer, que permet enviar PDUs entre dispositius.

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC0	PC10	ICMP		0.000	N	0	(edit)	(delete)
	Successful	PC1	PC8	ICMP		0.000	N	1	(edit)	(delete)
	Successful	PC2	PC10	ICMP		0.000	N	2	(edit)	(delete)
	Successful	Smartph...	PC10	ICMP		0.000	N	3	(edit)	(delete)

Figura 15: PDUs entre dispositius han estat rebudes correctament.

Com veiem, totes les connexions han estat exitoses. ■

Feu un ping des de un dels ordinadors a qualsevol altre. Aneu a Desktop, seleccioneu Command Prompt i, al terminal que s'obre, feu un ping a qualsevol dels altres ordinadors. Per veure que està passant cliqueu a Event List.

Exercici 3.6. *Explica el funcionament d'aquesta comanda i el que passa quan l'executes.*

Demostració. Quan escrivim ping seguit d'una adreça IP o un nom de domini, l'ordinador envia petits paquets de dades ICMP (en particular, ICMP Echo Request) al dispositiu destinatari i espera una resposta. El resultat mostra informació com:

- Si hi ha connectivitat amb el dispositiu destinatari.
- El temps que triga a arribar i tornar cada paquet (latència).
- El percentatge de pèrdues de paquets, si n'hi ha.

Si el dispositiu destinatari està disponible i configurat per respondre, aquest rebrà l'Echo Request. Automàticament, el dispositiu genera un ICMP Echo Reply, que torna cap al dispositiu que ha iniciat el ping. Quan el paquet Echo Reply torna al PC emissor, aquest registra el temps que ha trigat a anar i tornar. Si hi ha pèrdues de paquets, pot indicar problemes de connexió o configuració.

Event List és una manera visual d'entendre què està passant. Si obrim Event List i activem el mode Simulation, veurem com els paquets viatgen des del nostre ordinador fins al destinatari. Els paquets es mouen per la xarxa passant pels dispositius intermedis (switchos, routers) fins arribar al destinatari, el destinatari rep els paquets i genera respostes. ■

A l'Event List podeu veure tot el tràfic que s'està generant en la xarxa. Si feu un ping podeu analitzar el format de la trama Ethernet, el datagrama IP i el paquet ICMP.

Exercici 3.7. *Mostra el comportament i fes una captura de pantalla per veure què està passant.*

Demostració. Més enrere ja hem comprovat que les xarxes estan configurades correctament i la connexió entre els routers és correcta. Event List ens ajuda a veure què passa detalladament. Al final del procés, la finestra Event List mostra el següent:

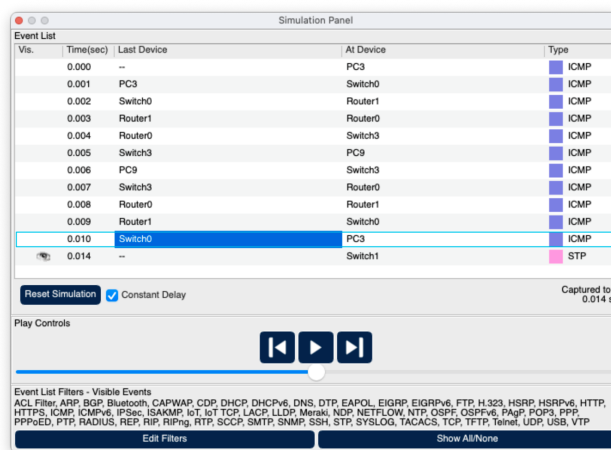


Figura 16: Event List.

Efectivament, la simulació representa un procés de ping complet entre PC₃ i PC₉, amb la sol·licitud (Echo Request) i la resposta (Echo Reply) circulant correctament per la xarxa, tot a partir de trames ICMP.

A destacar, el paquet es redirigeix per l'enllaç sèrie cap al segon router (Router0), seguint les taules d'en-caminament configurades; també, quan passa pel switch detecta que la destinació no és local i envia el paquet cap al router.

El fet que hi hagi un Echo Reply confirma que hi ha connectivitat entre PC₃ i PC₉. Si hi hagués algun problema de configuració o desconnexió, el procés es bloquejaria en algun punt, i no hi hauria resposta. ■

Un cop fet això, connecta el núvol a la teva xarxa a través del router de sortida (qualsevol dels dos routers utilitzats). Configureu NAT dinàmic en tots dos routers, assignant un pool de 2 IPs públiques a cada router. Aquestes IPs han de correspondre a la xarxa pública prèviament assignada al punt 4. A l'altra banda del núvol connecteu un servidor web. Necessitareu un protocol d'accés. Busqueu informació sobre Frame Relay i configureu la vostra connexió amb el núvol amb aquest protocol. Necessitareu establir rutes i/o un protocol d'en-caminament. Establiu rutes fixes amb la comanda `iproute` proporcionada per Cisco. Connecteu, amb l'ajuda dels professors la vostra xarxa amb el servidor.

Exercici 3.8. *Un cop connectats al servidor i, fent servir l'sniffer que subministra packet tracer identifiqueu els diferents protocols involucrats en la comunicació entre servidor i client.*

Demostració. Per configurar la connexió amb el núvol, connectarem el router al núvol amb un cable sèrie i activarem Frame Relay amb el DLCI corresponent. Assignarem una IP pública a la interfície sèrie del router i configurarem NAT dinàmic amb un pool de dues IPs públiques per traduir les adreces privades. A l'altra

banda del núvol, connectarem un servidor web amb una IP de la mateixa xarxa pública. Finalment, establim rutes fixes als routers perquè el trànsit arribi al núvol i al servidor, i verificarem la connectivitat amb pings i proves de navegació. Configurem una xarxa local al Router2 per al servidor:

```

1 Router(config)#ip dhcp pool SERVERNET
2 Router(dhcp-config)#network 192.168.3.0 255.255.255.0
3 Router(dhcp-config)#default-router 192.168.3.1
4 Router(dhcp-config)#dns-server 8.8.8.8
5 Router(dhcp-config)#exit
6 Router(config)#int Fa0/0
7 Router(config-if)#ip helper-address 192.168.3.1
8 Router(config-if)#ip address 192.168.3.1 255.255.255.0

```

D'altra banda, si intentem seguir l'esquema de la pràctica hauríem d'assignar IPs de la mateixa xarxa (la 161.116.95.0/24) a interfícies diferents del Router1, cosa que no podem fer. Optem per canviar la topologia de la xarxa a una semblant a la que tenim al document d'ajuda. Ens estalviem el problema de les IPs i no hauríem de tenir cap problema si configurem el routing estàtic correctament. Això sí, la connexió entre Router1 i Router0 es produirà a través del núvol. L'esquema seria el següent:

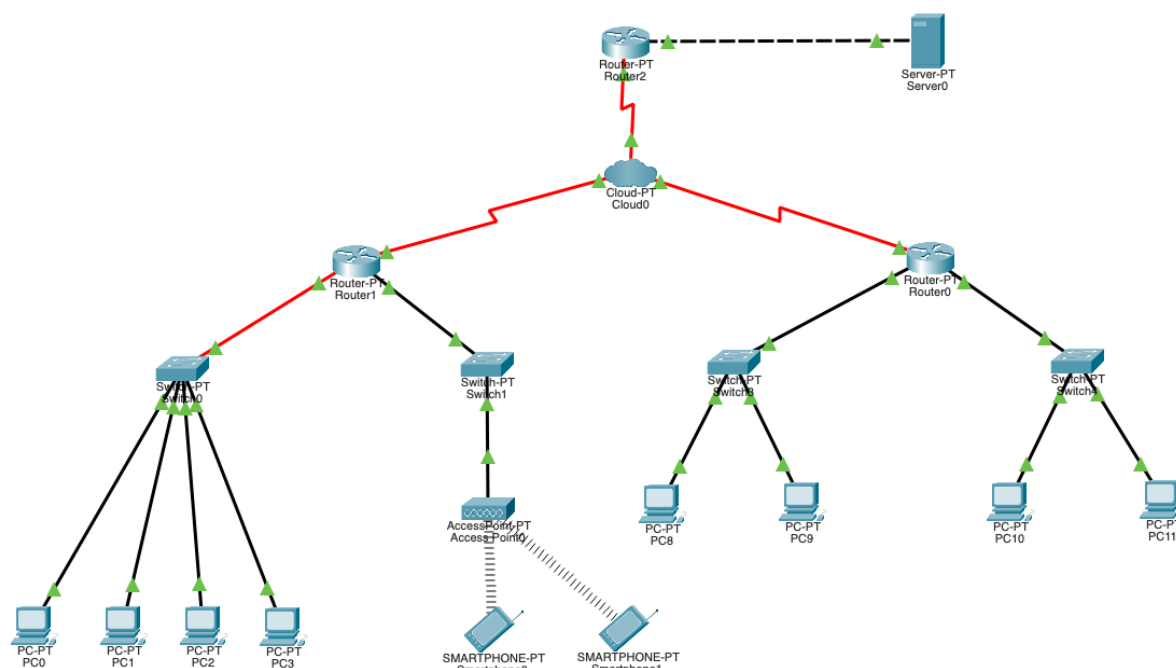


Figura 17: Esquema amb el núvol.

No entrarem en detalls perquè ho hem fet anteriorment, però hauríem de modificar les taules d'enrutament del Router0, Router1 i del Router2 per poder accedir a les seves respectives xarxes locals a través de la seva IP pública. Per exemple: Configurem ara el Frame Relay correctament:

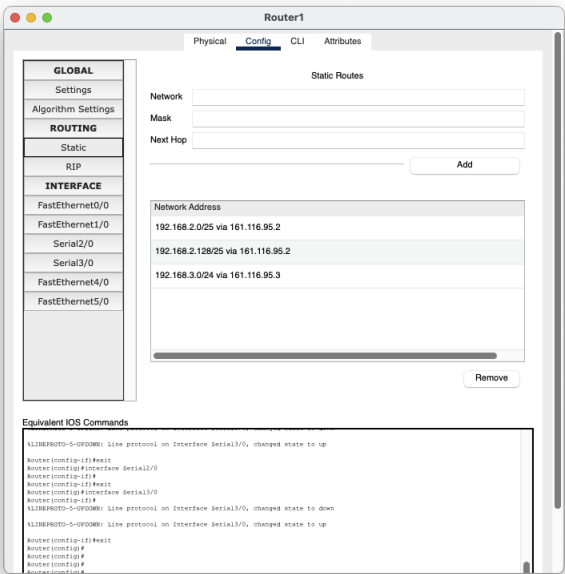


Figura 18: Més routing.

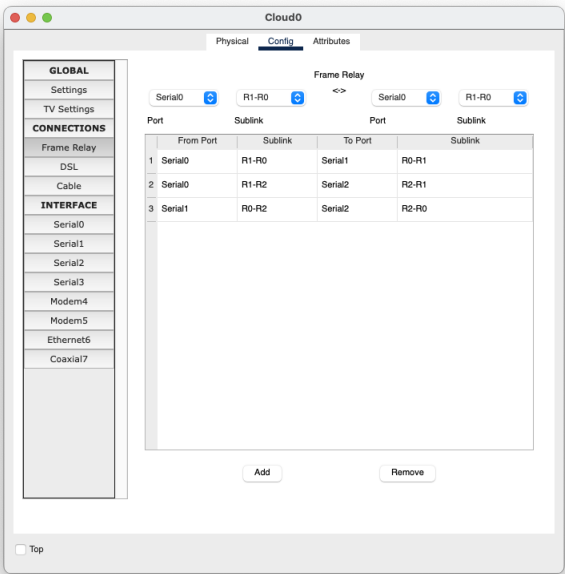


Figura 19: Frame Relay.

Si fem un ping del PC2 al PC8, per exemple, el resultat segueix essent satisfactori. També ho seria si fem un ping del PC2 al Servero:



Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC2	Server0	ICMP		0.000	N	0	(edit)	(delete)
	Successful	PC2	PC8	ICMP		0.000	N	1	(edit)	(delete)

Figura 20: Ping correcte.

Ara, configurem NAT. S'ha intentat poder configurar NAT a tots tres routers, o dos a dos, però no hi ha hagut manera. Adjuntarem a l'entrega un petit arxiu on hem estat fent proves amb diferents topologies, amb un NAT pool de dos adreces de la xarxa 161.116.95.0/24 per a cada router perquè es vegi que no és possible.

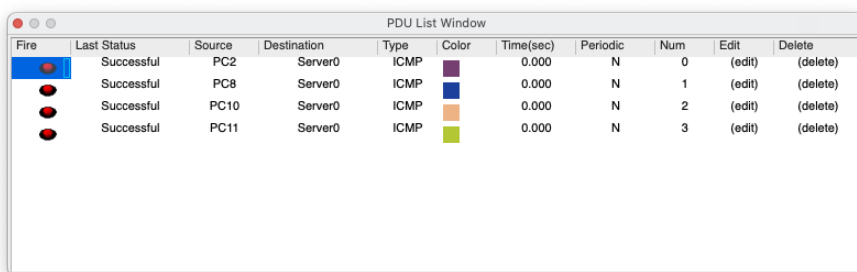
Amb el professor de pràctiques hem pactat configurar NAT a un sol router. Hem triat el Router2, pel que les comandes quedarien així:

```

1 Router(config)#int Fa0/0
2 Router(config-if)#ip nat inside
3 Router(config-if)#int Serial2/0
4 Router(config-if)#ip nat outside
5 Router(config-if)#exit
6 Router(config)#ip nat pool ROUTER2 161.116.95.4 161.116.95.5 netmask 255.255.255.0
7 Router(config)#access-list 1 permit 192.168.3.0 0.0.0.255
8 Router(config)#ip nat inside source list 1 pool ROUTER2

```

Podem veure que el resultat és satisfactori, ja que podem enviar un ping des de diversos PCs al servidor sense problemes:



Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC2	Server0	ICMP		0.000	N	0	(edit)	(delete)
	Successful	PC8	Server0	ICMP		0.000	N	1	(edit)	(delete)
	Successful	PC10	Server0	ICMP		0.000	N	2	(edit)	(delete)
	Successful	PC11	Server0	ICMP		0.000	N	3	(edit)	(delete)

Figura 21: Ping amb NAT correcte.

Per acabar, posem un sniffer entre el Switch3 i el Router0, per exemple. Fent un ping entre dispositius,

podem veure diferents formats de trama. Tenim, per exemple, els esdeveniments STP, DTP, CDP, ARP i ICMP (aquest últim ja l'hem esmentat anteriorment). Alguns formats de la trama ethernet serien:

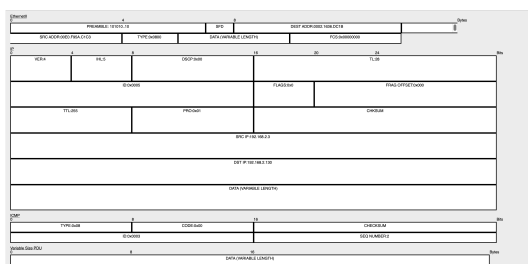


Figura 22: ICMP. DEST/SRC ADDR corresponen a les adreces de les interfícies.

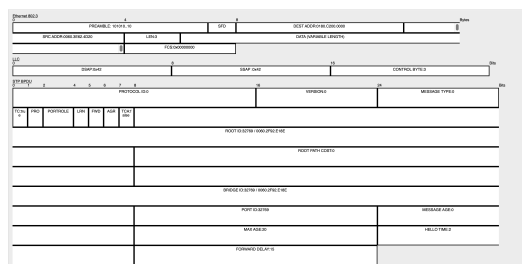


Figura 23: STP. És un protocol de capa 2 que evita bucles en xarxes amb múltiples switches.

A continuació, s'identifiquen els protocols que intervenen en la comunicació entre el servidor i el client, juntament amb una breu explicació del seu paper:

1. **Ethernet (Capa 2 - Enllaç de dades):** Aquest protocol encapsula els paquets IP en trames Ethernet per enviar-los a través de la xarxa local (LAN). També gestiona les adreces MAC dels dispositius per assegurar que el paquet arribi correctament al destí dins de la LAN.
2. **IP (Capa 3 - Xarxa):** El protocol IP proporciona les adreces úniques necessàries per identificar els dispositius a la xarxa. En aquest exercici, s'utilitzen adreces privades dins de la LAN, que són traduïdes a adreces públiques mitjançant NAT per poder comunicar-se amb el servidor.
3. **NAT (Capa 3 - Xarxa):** El *Network Address Translation* permet que les adreces IP privades dels clients siguin traduïdes en adreces IP públiques del *pool* assignat al router. Això facilita la comunicació entre dispositius amb adreces privades i el servidor extern a través d'una xarxa pública.
4. **TCP (Capa 4 - Transport):** El protocol TCP garanteix una comunicació fiable entre el client i el servidor. Divideix les dades en segments, retransmet segments perduts i assegura que els segments es rebin de manera ordenada. Aquest protocol és fonamental per establir i mantenir la connexió durant la transferència de dades.
5. **Frame Relay (Capa 2 - Enllaç de dades):** Aquest protocol s'utilitza per establir la connexió entre els routers i el núvol. Permet una transferència eficient de dades encapsulant la informació i assignant identificadors DLCI per al ruteig correcte dins del núvol.
6. **ICMP (Capa 3 - Xarxa):** Quan es realitzen proves de connectivitat amb el servidor mitjançant *ping*, s'utilitzen paquets ICMP. Aquests inclouen sol·licituds (*Echo Request*) i respostes (*Echo Reply*) per verificar si la connexió està activa.
7. **STP (Capa 2 - Enllaç de dades):** El *Spanning Tree Protocol* evita la formació de bucles en xarxes amb múltiples switches i assegura un camí òptim per al trànsit.
8. **CDP (Capa 2 - Enllaç de dades):** El *Cisco Discovery Protocol* permet que dispositius Cisco compartei-

xin informació sobre connexions directes, facilitant la gestió de la xarxa.

9. **DTP (Capa 2 - Enllaç de dades):** El *Dynamic Trunking Protocol* automatitza la configuració de ports com a access o trunk en switches, simplificant la gestió de VLANs.
10. **ARP (Capa 2 - Enllaç de dades):** L'*Address Resolution Protocol* tradueix adreces IP en adreces MAC per facilitar la comunicació dins de la mateixa xarxa local. ■

Exercici 3.9. *A partir d'aquesta informació, compareu els protocols i els resultats amb la tasca feta en la primera part de la pràctica.*

Demostració. Pel que fa als protocols utilitzats a la primera part:

1. HTTP: Per enviar dades a ThingSpeak. S'ha implementat manualment primer i després utilitzant la llibreria ThingSpeak.h, simplificant el procés.
2. Wi-Fi: S'ha establert la comunicació mitjançant el protocol Wi-Fi, utilitzant el dispositiu ESP8266 per obtenir i transmetre les dades RSSI.

Pel que fa als protocols utilitzats a la segona part:

1. DHCP: Assignació automàtica d'adreces IP a dispositius de la xarxa.
2. ICMP (Ping): Per verificar la connectivitat entre dispositius.
3. NAT: Per traduir adreces IP privades en IP públiques i permetre la comunicació amb el servidor web a través d'una xarxa pública.
4. Frame Relay: Utilitzat per connectar routers mitjançant el núvol.

Fem una breu comparació. A la primera part, els protocols es limiten al Wi-Fi i HTTP. A la segona part, es treballa amb una gamma més àmplia de protocols (DHCP, NAT, Frame Relay), necessaris per gestionar una infraestructura de xarxa més completa. Això és perquè els objectius i els resultats de la primera i segona part són inherentment diferents: a la primera part ens hem centrat en la configuració d'un dispositiu client amb l'objectiu d'enviar dades a ThingSpeak, mentre que a la segona part hem treballat molt més a nivell de xarxa. I és que les capes OSI que hem treballat han estat una mica diferents en aquestes dues parts.

En la primera part de la pràctica, la comunicació s'ha centrat en diverses capes del model OSI. A la capa física i d'enllaç, hem connectat l'ESP8266 a la xarxa Wi-Fi; després, a la capa de xarxa, s'ha gestionat l'adreçament IP per enviar dades a ThingSpeak. A la capa de transport, hem utilitzat TCP per garantir la fiabilitat, i a la capa d'aplicació, hem fet servir HTTP per la comunicació amb el servidor. A la segona part, però, s'ha treballat més en capes intermèdies: la de xarxa i la d'enllaç les hem usat amb protocols com Frame Relay, NAT i ICMP. També hem configurat DHCP per a l'assignació d'IPs.

Pel que fa als resultats:

- A la primera part, el resultat ha sigut que s'ha aconseguit mesurar la intensitat del senyal Wi-Fi (RSSI) i enviar-la al servidor ThingSpeak. Amb el codi manual, vam haver de fer una implementació més detallada, com construir les peticions HTTP.

- A la segona part hem implementat solucions més complexes, com la connexió entre subxarxes, subnetting i un servidor remot, amb proves de ping per verificar la connectivitat. ■

4

CONCLUSIONS

Aquesta pràctica té com a objectiu principal entendre el funcionament del protocol TCP/IP i posar en pràctica les seves aplicacions en un sistema de transferència de dades. Per aconseguir-ho, hem treballat en dos escenaris diferents: el primer enfocat a la implementació física amb un dispositiu ESP8266, i el segon, en un entorn de simulació amb Packet Tracer.

A la primera part, ens hem centrat en la creació d'un canal recol·lector de dades. Hem configurat l'ESP8266 per connectar-lo a una xarxa Wi-Fi i enviar dades (concretament la intensitat del senyal RSSI) a la plataforma ThingSpeak. Hem implementat aquest procés tant manualment, construint peticions HTTP, com utilitzant una llibreria que simplifica les tasques repetitives.

La segona part ens ha portat a simular una xarxa més complexa amb Packet Tracer. Hem dissenyat i configurat diferents dispositius, com routers, switches i PCs, assignant IPs i gestionant protocols com DHCP, NAT i Frame Relay. També hem experimentat amb subxarxes, connectant diverses xarxes locals a un servidor extern a través d'una xarxa pública.

Amb aquesta pràctica, hem explorat com els protocols i la infraestructura de xarxa s'adapten a necessitats diverses, des d'una configuració senzilla fins a una arquitectura més robusta i interconnectada. Això ens ha permès veure en detall com funciona la comunicació a nivell de xarxes i quins reptes poden sorgir en cada context.